Lab 02

TITLE

Hands-On Git and Version Control

Name:

Irza Hameed

Registration number:

(2023-BSCS-44)

Section:

A

GitHub username:

irzahameed

Lab 2: Hands-On Git and Version Control

Estimated duration: 3 hours

Submission repository: DevOps

Submission directory: Labs/Lab02

Screenshot directory: Labs/Lab02/screenshots

Complete the practical work in the repository required by each task. Submit all evidence to your GitHub repository named exactly DevOps using the directory structure shown below.

```
DevOps/
└── Labs/
    └── Lab02/
        ├── README.md
        ├── notes.txt
        ├── main.py
        ├── Lab2_Solution.pdf
        └── screenshots/
            ├── git_installation.png
            ├── repo_private.png
            └── ...
```

Navigation: Click any task below to jump to its instructions.

Task List

Getting Started

Task 1: Create Private GitHub Repository

Task 2: Connect Repository via SSH

Task 3: Configure Git Username and Email

Task 4: Explore the .git Folder

Task 5: Local Repository Management

Task 6: File Status & Staging

Task 7: Branch Creation Using GitHub GUI

Task 8: Branch Creation and Push Using Git Bash

Task 9: Branching & Merging

Task 10: Pull Request and Branch Review (GitHub GUI)

Task 11: Detailed Branch Strategy (Develop/Staging)

Task 12: Code Review Workflow

Task 13: Branch Cleanup Best Practices

Bonus Task: Simulated Team Collaboration

Exam Evaluation Questions

Mandatory Screenshot and Autograding Rules

Required Automated-Grading Filenames

The current Lab 2 grader checks these 37 files:

git_installation.png

repo_private.png

ssh_keygen.png

github_sshkey.png

ssh_clone.png

git_identity.png

git_config_list.png

git_folder.png

delete_git.png

git_init.png

first_commit.png

first_push.png

status1.png

commit_notes.png

bugfix_branch_gui.png

bugfix_branch_local.png

feature_db_branch.png

branch_create.png

feature_commit.png

merge.png

push_branches.png

pr_creation.png

pr_merge.png

branch_delete.png

branch_strategy.png

branch_merges.png

final_merge.png

pr_create_details.png

pr_assigned_reviewer.png

pr_request_changes.png

pr_rejected.png

pr_updated_with_commits.png

pr_merge_confirm.png

pr_merged.png

pr_branch_deleted.png

remote_branch_deleted.png

remote_branch_delete_cmd.png

Each of these 37 checks has equal weight. Bonus and additional exam-practice screenshots may be reviewed separately and are not included in these 37 automated checks.

Getting Started

1. Install Git on your PC.

- Download Git for Windows, Mac, or Linux
- Save the evidence as Labs/Lab02/screenshots/git_installation.png.

ounto-server [Running] - Oracle VirtualBox

Machine View Input Devices Help

```
  IPv4 address for enp0s3: 10.0.2.15
  IPv6 address for enp0s3: fd17:625c:f037:2:a00:27ff:fe63:b3a4

 * Canonical Workshop gives developers fast, composable, reproducible, and
   secure developer environments that are perfect for agentic workflows.

   https://ubuntu.com/workshop

Expanded Security Maintenance for Applications is not enabled.

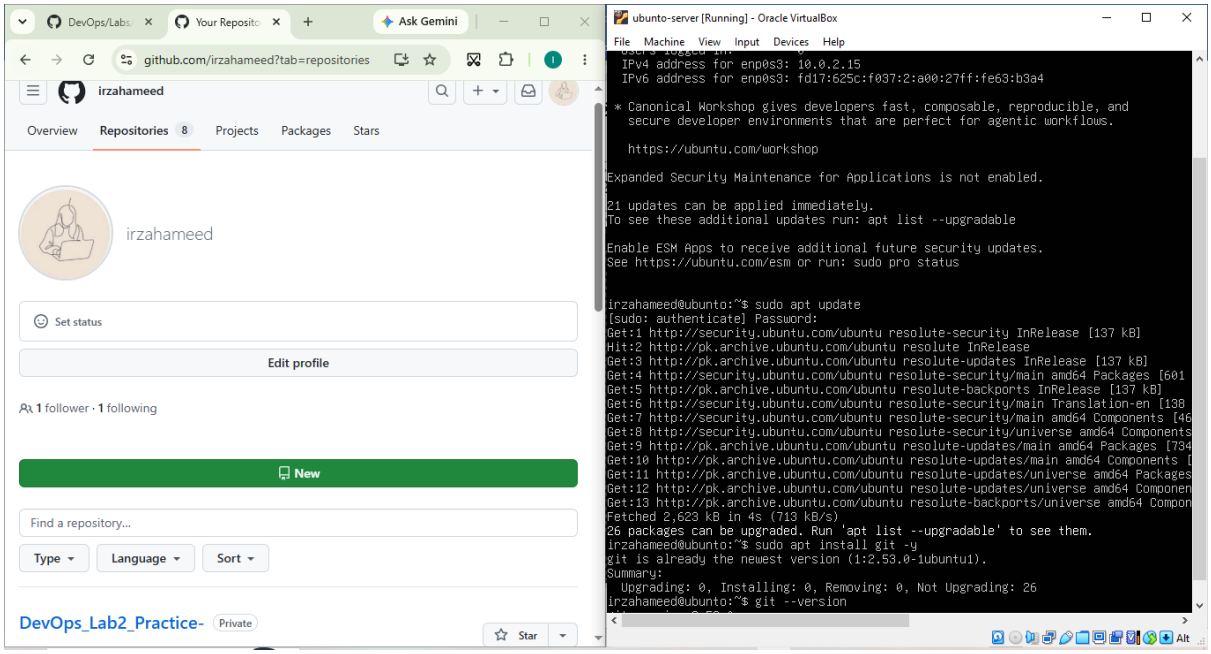
21 updates can be applied immediately.
To see these additional updates run: apt list --upgradable

Enable ESM Apps to receive additional future security updates.
See https://ubuntu.com/esm or run: sudo pro status


irzahameed@ubunto:~$ sudo apt update
[sudo: authenticate] Password:
Get:1 http://security.ubuntu.com/ubuntu resolute-security InRelease [137 kB]
Hit:2 http://pk.archive.ubuntu.com/ubuntu resolute InRelease
Get:3 http://pk.archive.ubuntu.com/ubuntu resolute-updates InRelease [137 kB]
Get:4 http://security.ubuntu.com/ubuntu resolute-security/main amd64 Packages [601 kB]
Get:5 http://pk.archive.ubuntu.com/ubuntu resolute-backports InRelease [137 kB]
Get:6 http://security.ubuntu.com/ubuntu resolute-security/main Translation-en [138 kB]
Get:7 http://security.ubuntu.com/ubuntu resolute-security/main amd64 Components [46.7 kB]
Get:8 http://security.ubuntu.com/ubuntu resolute-security/universe amd64 Components [55.5 kB]
Get:9 http://pk.archive.ubuntu.com/ubuntu resolute-updates/main amd64 Packages [734 kB]
Get:10 http://pk.archive.ubuntu.com/ubuntu resolute-updates/main amd64 Components [110 kB]
Get:11 http://pk.archive.ubuntu.com/ubuntu resolute-updates/universe amd64 Packages [323 kB]
Get:12 http://pk.archive.ubuntu.com/ubuntu resolute-updates/universe amd64 Components [201 kB]
Get:13 http://pk.archive.ubuntu.com/ubuntu resolute-backports/universe amd64 Components [1,056 B]
Fetched 2,623 kB in 4s (713 kB/s)
26 packages can be upgraded. Run 'apt list --upgradable' to see them.
irzahameed@ubunto:~$ sudo apt install git -y
git is already the newest version (1:2.53.0-1ubuntu1).
Summary:
  Upgrading: 0, Installing: 0, Removing: 0, Not Upgrading: 26
irzahameed@ubunto:~$ git --version
git version 2.53.0
irzahameed@ubunto:~$
```

Task 1: Create Private GitHub Repository

1. Create a new private repository named exactly DevOps_Lab2_Practice on GitHub.
2. Take a split-screen screenshot showing the repository is private and your required terminal prompt. Save it as Labs/Lab02/screenshots/repo_private.png.



Task 2: Connect Repository via SSH

1. Generate a new SSH key using PowerShell:

ssh-keygen -t ed25519 -C "your_email@example.com"

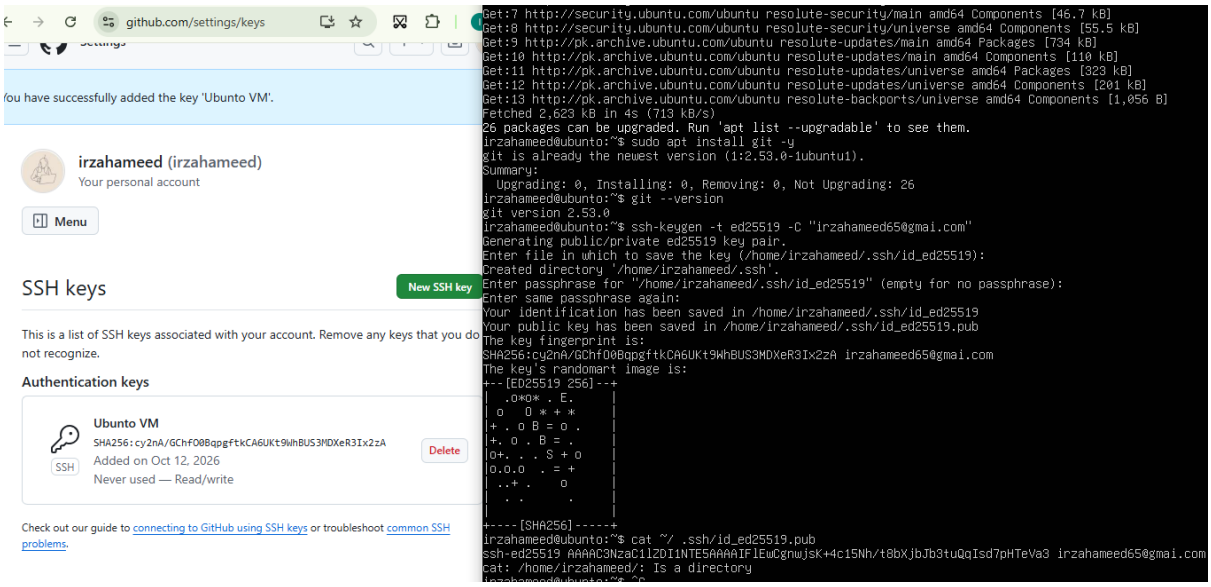
cat ~/.ssh/id_ed25519.pub

- Save it as Labs/Lab02/screenshots/ssh_keygen.png. Do not show the private key contents.

```
irzahameed@ubunto:~$ git --version
git version 2.53.0
irzahameed@ubunto:~$ ssh-keygen -t ed25519 -C "irzahameed65@gmai.com"
Generating public/private ed25519 key pair.
Enter file in which to save the key (/home/irzahameed/.ssh/id_ed25519):
Created directory '/home/irzahameed/.ssh'.
Enter passphrase for "/home/irzahameed/.ssh/id_ed25519" (empty for no passphrase):
Enter same passphrase again:
Your identification has been saved in /home/irzahameed/.ssh/id_ed25519
Your public key has been saved in /home/irzahameed/.ssh/id_ed25519.pub
The key fingerprint is:
SHA256:cy2nA/GChfO0BqpgftkCA6UKt9WhBUS3MDXeR3Ix2zA irzahameed65@gmai.com
The key's randomart image is:
+--[ED25519 256]--+
|  .o*o* . E.     |
| o   O * + *     |
|+ . o B = o .    |
|+. o . B = .     |
|o+. . . S + o    |
|o.o.o  . = +     |
| ..+ .    o      |
|  . .      .     |
|                 |
+----[SHA256]-----+
irzahameed@ubunto:~$ cat ~/ .ssh/id_ed25519.pub
ssh-ed25519 AAAAC3NzaC1lZDI1NTE5AAAAIFlEwCgnwjsK+4c15Nh/t8bXjbJb3tuQqIsd7pHTeVa3 irzahameed65@gmai.com
cat: /home/irzahameed/: Is a directory
irzahameed@ubunto:~$
```

2. Add your SSH public key to GitHub (Settings > SSH and GPG keys).

- Save a split-screen screenshot as Labs/Lab02/screenshots/github_sshkey.png.



3. Clone your DevOps_Lab2_Practice repository using SSH.

git clone git@github.com:<yourusername>/DevOps_Lab2_Practice.git

- Save it as Labs/Lab02/screenshots/ssh_clone.png.

```
irzahameed@ubunto:~$ ssh -T git@github.com
The authenticity of host 'github.com (20.207.73.82)' can't be established.
ED25519 key fingerprint is: SHA256:+DiY3wvvV6TuJJhbpZisF/zLDA0zPMSvHdkr4UvCOqU
This key is not known by any other names.
Are you sure you want to continue connecting (yes/no/[fingerprint])? yes
Warning: Permanently added 'github.com' (ED25519) to the list of known hosts.
Hi irzahameed! You've successfully authenticated, but GitHub does not provide shell access.
irzahameed@ubunto:~$ cd ~
irzahameed@ubunto:~$ git clone git@github.com:irzahameed65@gmail.com/DevOps_Lab2_Practice.git
Cloning into 'DevOps_Lab2_Practice'...
fatal: remote error:
 irzahameed65@gmail.com/DevOps_Lab2_Practice is not a valid repository name
Visit https://support.github.com/ for help
irzahameed@ubunto:~$ git clone git@github.com:irzahameed65@gmail.com/DevOps_Lab2_Practice.git
Cloning into 'DevOps_Lab2_Practice'...
fatal: remote error:
 irzahameed65@gmail.com/DevOps_Lab2_Practice is not a valid repository name
Visit https://support.github.com/ for help
irzahameed@ubunto:~$ git clone git@github.com:irzahameed/Devops_Lab2_Practice.git
Cloning into 'Devops_Lab2_Practice'...
warning: You appear to have cloned an empty repository.
irzahameed@ubunto:~$
```

Task 3: Configure Git Username and Email

1. Set up your Git identity (this ensures all commits are linked to you):

git config --global user.name "Your Name"

git config --global user.email "your_email@example.com"

- Save a screenshot of your terminal as git_identity.png.

```
irzahameed@ubunto:~$ git config --global user.name "Irza Hameed"
irzahameed@ubunto:~$ git config --global user.email "irzahameed65@gmail.com"
irzahameed@ubunto:~$
```

2. Verify your configuration:

git config --list

- Screenshot as git_config_list.png.

```
irzahameed@ubunto:~$ git config --list
user.name=Irza Hameed
user.email=irzahameed65@gmail.com
irzahameed@ubunto:~$
```

Task 4: Explore the .git Folder

1. Navigate into your cloned repository folder.
2. Show hidden files and locate the .git directory.
3. Explore what's inside using:

ls -a .git

- Take a screenshot as git_folder.png.

```
irzahameed@ubunto:~$ cd ~/DevOps_Lab2_Practice
irzahameed@ubunto:~/DevOps_Lab2_Practice$ ls -a
.  ..  .git
irzahameed@ubunto:~/DevOps_Lab2_Practice$ ls -a .git
.  ..  HEAD  config  description  hooks  info  objects  refs
irzahameed@ubunto:~/DevOps_Lab2_Practice$
```

4. Note: This folder contains history, logs, and config for your repository. Do not modify these files manually.

Task 5: Local Repository Management

1. Delete the existing .git folder from your cloned repo using Git Bash:

rm -rf .git

git status ==> error why?

- Take a screenshot as delete_git.png.

```
irzahameed@ubunto:~/DevOps_Lab2_Practice$ cd ~/DevOps_Lab2_Practice
irzahameed@ubunto:~/DevOps_Lab2_Practice$ rm -rf .git
irzahameed@ubunto:~/DevOps_Lab2_Practice$ git status
fatal: not a git repository (or any of the parent directories): .git
irzahameed@ubunto:~/DevOps_Lab2_Practice$
```

2. Re-initialize the local git repository:

git init

- Save a screenshot as git_init.png.

```
irzahameed@ubunto:~/DevOps_Lab2_Practice$ git init
hint: Using 'master' as the name for the initial branch. This default branch name
hint: will change to "main" in Git 3.0. To configure the initial branch name
hint: to use in all of your new repositories, which will suppress this warning,
hint: call:
hint:
hint:   git config --global init.defaultBranch <name>
hint:
hint: Names commonly chosen instead of 'master' are 'main', 'trunk' and
hint: 'development'. The just-created branch can be renamed via this command:
hint:
hint:   git branch -m <name>
hint:
hint: Disable this message with "git config set advice.defaultBranchName false"
Initialized empty Git repository in /home/irzahameed/DevOps_Lab2_Practice/.git/
irzahameed@ubunto:~/DevOps_Lab2_Practice$
```

3. Add a file named README.md and commit it:

echo "# Lab02 Git Practice" >> README.md

git add README.md

git commit -m "Initial commit"

- Save a screenshot as first_commit.png.

```
Initialized empty Git repository in /home/irzahameed/DevOps_Lab2_Practice/.git/
irzahameed@ubunto:~/DevOps_Lab2_Practice$ echo "# Lab02 Git Practice" >> README.md
irzahameed@ubunto:~/DevOps_Lab2_Practice$ git add README.md
irzahameed@ubunto:~/DevOps_Lab2_Practice$ git commit -m "Initial commit"
[master (root-commit) 8e39214] Initial commit
 1 file changed, 1 insertion(+)
 create mode 100644 README.md
irzahameed@ubunto:~/DevOps_Lab2_Practice$
```

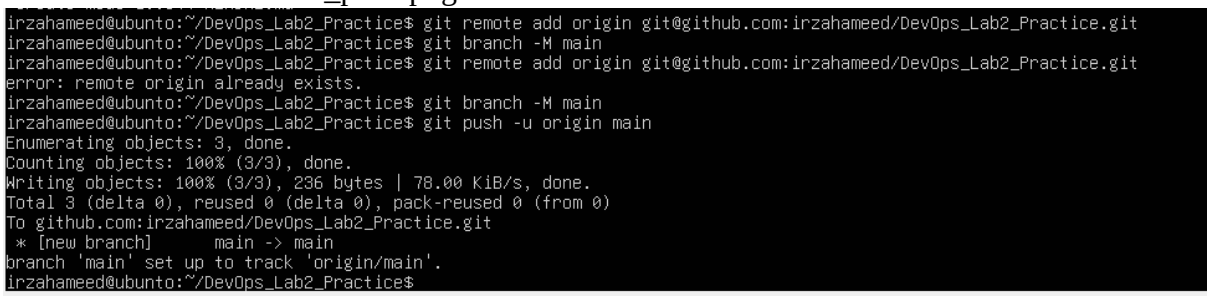
4. Connect your local repo to GitHub and push:

git remote add origin git@github.com:<yourusername>/DevOps_Lab2_Practice.git

git branch -M main

git push -u origin main

- Save a screenshot as first_push.png.



Task 6: File Status & Staging

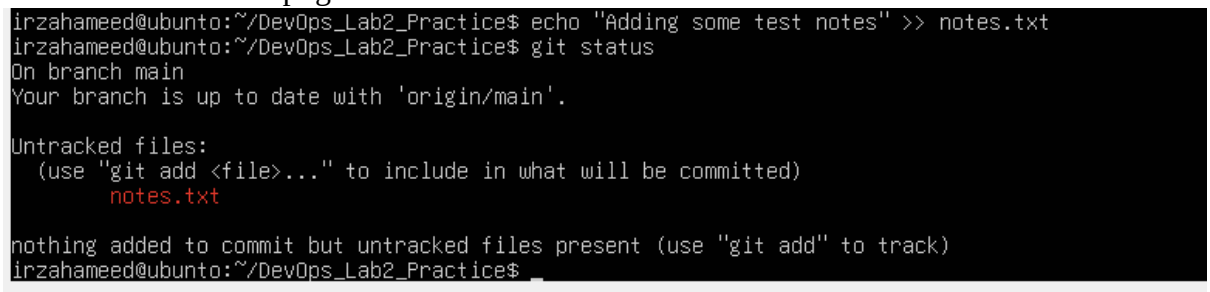
1. Create a new file notes.txt and write a note.

echo "Adding some test notes" >> notes.txt

2. Check status:

git status

- Screenshot as status1.png.



3. Stage and commit:

git add notes.txt

git commit -m "Add notes.txt"

- Screenshot as commit_notes.png.



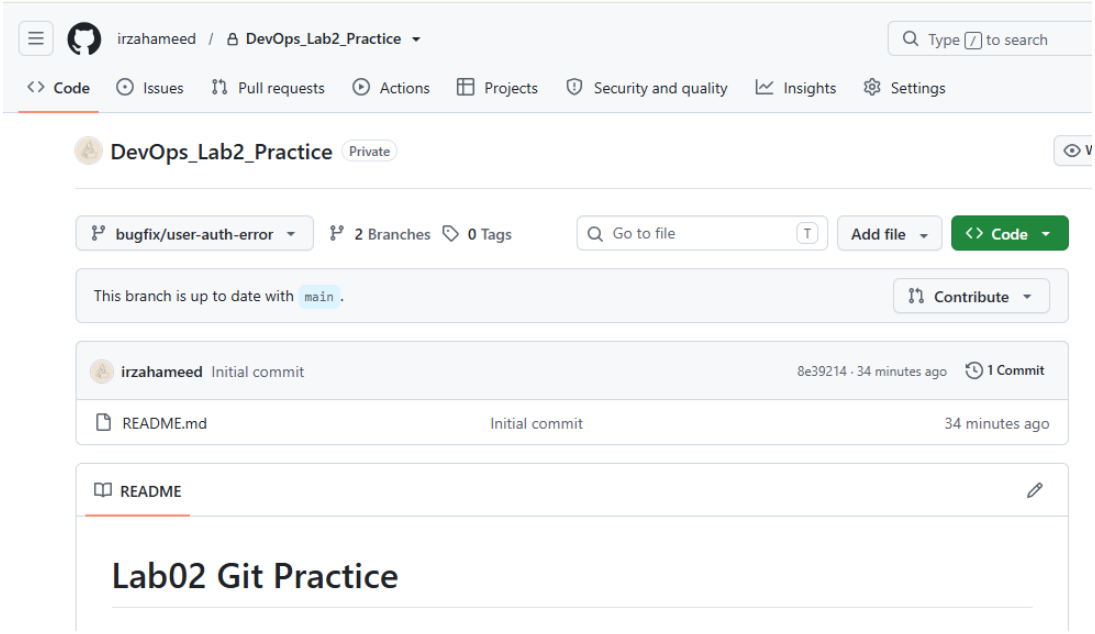
- Edit notes.txt and repeat status/commit steps.

Task 7: Branch Creation Using GitHub GUI

1. On GitHub (web interface), create a branch named bugfix/user-auth-error.
2. Pull the branch to your local repository to sync.

git pull origin bugfix/user-auth-error

- Take a screenshot of branch creation in the GUI as bugfix_branch_gui.png.



- Take a screenshot of your local git branch list after pulling as bugfix_branch_local.png.

```
irzahameed@ubunto:~/DevOps_Lab2_Practice$ git pull origin bugfix/user-auth-error
From github.com:irzahameed/DevOps_Lab2_Practice
 * branch            bugfix/user-auth-error -> FETCH_HEAD
 * [new branch]      bugfix/user-auth-error -> origin/bugfix/user-auth-error
Already up to date.
irzahameed@ubunto:~/DevOps_Lab2_Practice$ git branch -a
* main
  remotes/origin/HEAD -> origin/main
  remotes/origin/bugfix/user-auth-error
  remotes/origin/main
irzahameed@ubunto:~/DevOps_Lab2_Practice$
```

Task 8: Branch Creation and Push Using Git Bash

1. Create a branch named feature/db-connection using Git Bash:

git checkout -b feature/db-connection

2. Push the branch to the remote repository:

git push origin feature/db-connection

- Take a screenshot of the branch creation and push as feature_db_branch.png.

```
irzahameed@ubunto:~/DevOps_Lab2_Practice$ git checkout -b feature/db-connection
Switched to a new branch 'feature/db-connection'
irzahameed@ubunto:~/DevOps_Lab2_Practice$ git push origin feature/db-connection
Enumerating objects: 4, done.
Counting objects: 100% (4/4), done.
Compressing objects: 100% (2/2), done.
Writing objects: 100% (3/3), 298 bytes | 149.00 KiB/s, done.
Total 3 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)
remote:
remote: Create a pull request for 'feature/db-connection' on GitHub by visiting:
remote:      https://github.com/irzahameed/DevOps_Lab2_Practice/pull/new/feature/db-connection
remote:
To github.com:irzahameed/DevOps_Lab2_Practice.git
 * [new branch]      feature/db-connection -> feature/db-connection
irzahameed@ubunto:~/DevOps_Lab2_Practice$
```

Task 9: Branching & Merging

1. Create and switch to a branch feature-1:

git checkout -b feature-1

- Screenshot as branch_create.png.

```
irzahameed@ubunto:~/DevOps_Lab2_Practice$ git checkout main
Already on 'main'
Your branch is ahead of 'origin/main' by 2 commits.
  (use "git push" to publish your local commits)
irzahameed@ubunto:~/DevOps_Lab2_Practice$ git checkout -b feature-1
Switched to a new branch 'feature-1'
irzahameed@ubunto:~/DevOps_Lab2_Practice$
```

2. Modify main.py (add a function) and commit.

git add main.py

git commit -m "Add new function to main.py"

- Screenshot as feature_commit.png.

```
irzahameed@ubunto:~/DevOps_Lab2_Practice$ echo "def hello():" >> main.py
irzahameed@ubunto:~/DevOps_Lab2_Practice$ echo "  print('Hello from feature-1')" >> main.py
irzahameed@ubunto:~/DevOps_Lab2_Practice$ git add main.py
irzahameed@ubunto:~/DevOps_Lab2_Practice$ git commit -m "Add new function to main.py"
[feature-1 9b73f4b] Add new function to main.py
 1 file changed, 2 insertions(+)
irzahameed@ubunto:~/DevOps_Lab2_Practice$ _
```

Lab 2 DevOps - De

3. Switch back to main and merge:

git checkout main

git merge feature-1

- Screenshot as merge.png.

```
irzahameed@ubunto:~/DevOps_Lab2_Practice$ git checkout main
Switched to branch 'main'
Your branch is ahead of 'origin/main' by 2 commits.
  (use "git push" to publish your local commits)
irzahameed@ubunto:~/DevOps_Lab2_Practice$ git merge feature-1
Updating ff101a7..9b73f4b
Fast-forward
 main.py | 2 ++
 1 file changed, 2 insertions(+)
irzahameed@ubunto:~/DevOps_Lab2_Practice$
```

Merging directly into main from your local terminal is widely considered a risky anti-pattern, especially on a team. In professional software development, using a Pull Request (PR) or Merge Request (MR) is the gold standard.

4. Push all branches:

git push origin main

git push origin feature-1

- Screenshot as push_branches.png.

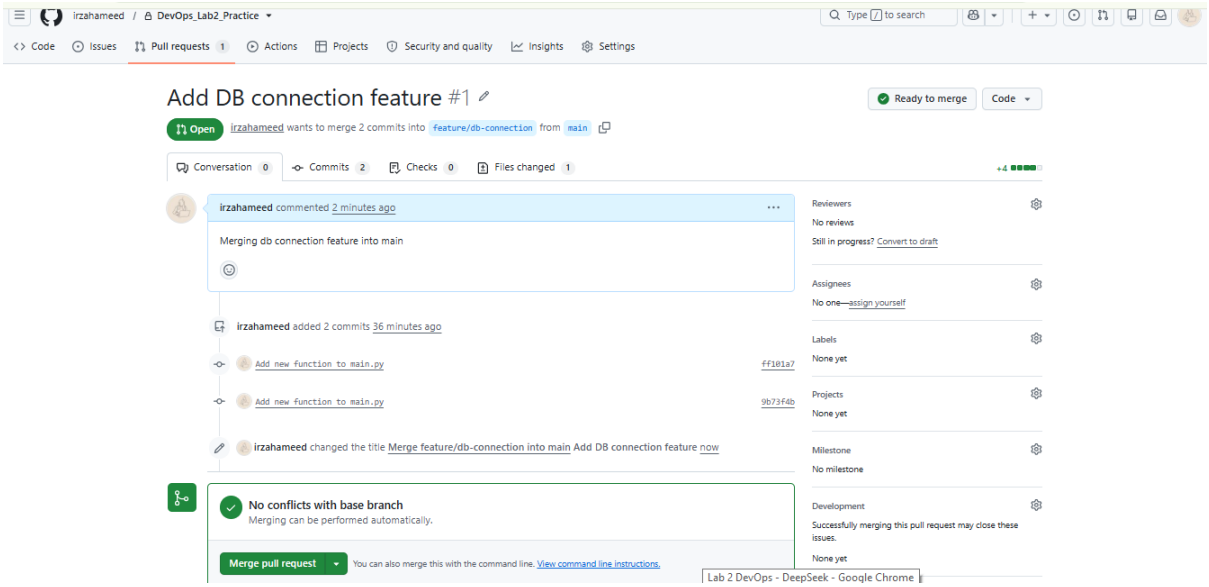
```
irzahameed@ubunto:~/DevOps_Lab2_Practice$ git push origin main
Enumerating objects: 7, done.
Counting objects: 100% (7/7), done.
Compressing objects: 100% (5/5), done.
Writing objects: 100% (6/6), 699 bytes | 87.00 KiB/s, done.
Total 6 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)
To github.com:irzahameed/DevOps_Lab2_Practice.git
   8e39214..9b73f4b  main -> main
irzahameed@ubunto:~/DevOps_Lab2_Practice$ git push origin feature-1
Total 0 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)
remote:
remote: Create a pull request for 'feature-1' on GitHub by visiting:
remote:      https://github.com/irzahameed/DevOps_Lab2_Practice/pull/new/feature-1
remote:
To github.com:irzahameed/DevOps_Lab2_Practice.git
 * [new branch]      feature-1 -> feature-1
irzahameed@ubunto:~/DevOps_Lab2_Practice$
```

Task 10: Pull Request and Branch Review (GitHub GUI)

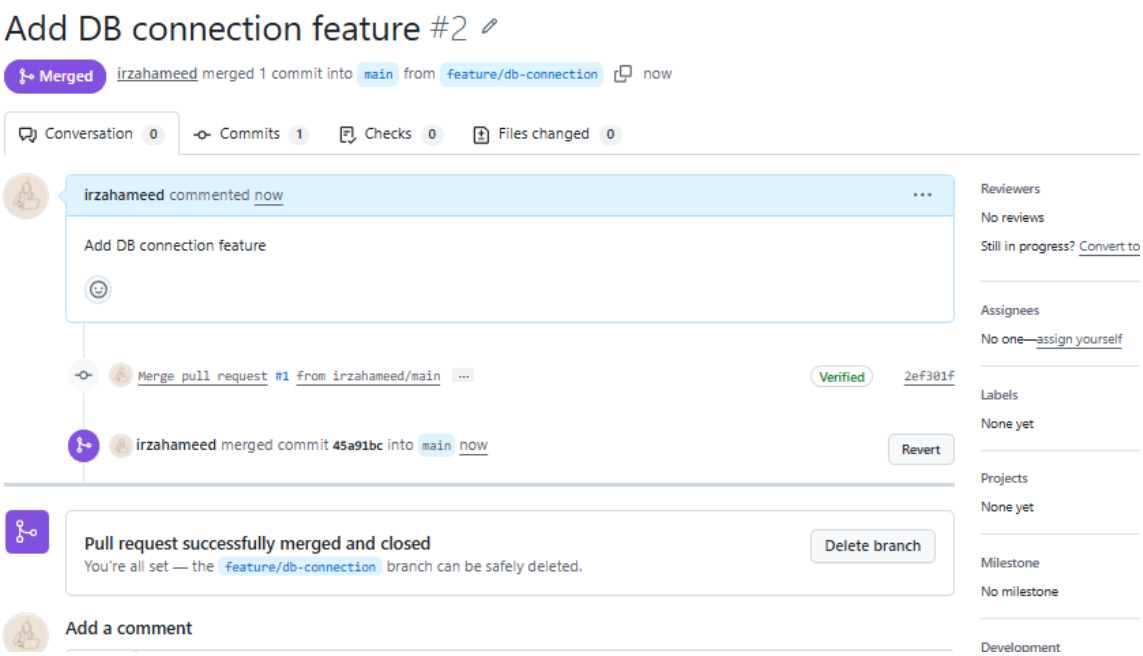
1. On GitHub, create a Pull Request from the branch feature/db-connection to main.
2. Review the Pull Request and merge it using the GitHub GUI.
3. After merging, delete the feature/db-connection branch using the GitHub GUI.

- Take a screenshot of:

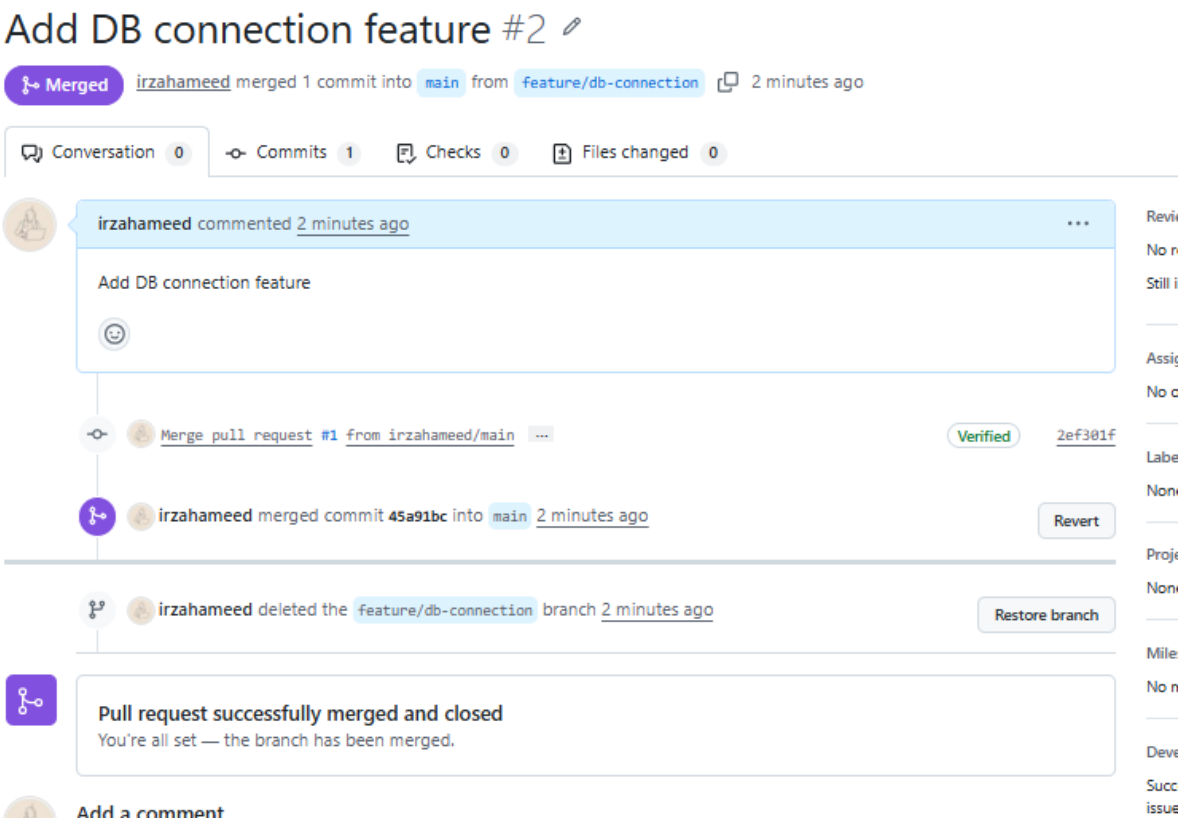
➢ The Pull Request creation (pr_creation.png)



- The Pull Request review and merge (pr_merge.png)



- The branch deletion (branch_delete.png)



Task 11: Detailed Branch Strategy (Develop/Staging)

1. Create the following branches to simulate a professional branching strategy:
- develop
- staging
- feature/*
- bugfix/*
2. Example workflow:
- Developers work on feature/* and bugfix/* branches.
- Merge into develop after completion.
- develop is merged into staging for testing.
- Finally, staging is merged into main (production).
3. Document the workflow with screenshots:
- Branch list (branch_strategy.png)

```
irzahameed@ubunto:~/DevOps_Lab2_Practice$ git checkout -b develop
Switched to a new branch 'develop'
irzahameed@ubunto:~/DevOps_Lab2_Practice$ git push origin develop
Total 0 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)
remote:
remote: Create a pull request for 'develop' on GitHub by visiting:
remote:      https://github.com/irzahameed/DevOps_Lab2_Practice/pull/new/develop
remote:
To github.com:irzahameed/DevOps_Lab2_Practice.git
 * [new branch]      develop -> develop
irzahameed@ubunto:~/DevOps_Lab2_Practice$ git checkout -b staging
Switched to a new branch 'staging'
irzahameed@ubunto:~/DevOps_Lab2_Practice$ git push origin staging
Total 0 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)
remote:
remote: Create a pull request for 'staging' on GitHub by visiting:
remote:      https://github.com/irzahameed/DevOps_Lab2_Practice/pull/new/staging
remote:
To github.com:irzahameed/DevOps_Lab2_Practice.git
 * [new branch]      staging -> staging
irzahameed@ubunto:~/DevOps_Lab2_Practice$ git branch -a
  develop
  feature-1
  feature/db-connection
  main
* staging
  remotes/origin/HEAD -> origin/main
  remotes/origin/bugfix/user-auth-error
  remotes/origin/develop
  remotes/origin/feature-1
  remotes/origin/feature/db-connection
  remotes/origin/main
  remotes/origin/staging
irzahameed@ubunto:~/DevOps_Lab2_Practice$
```

- Merges into develop and staging (branch_merges.png)

```
irzahameed@ubunto:~/DevOps_Lab2_Practice$ git checkout -b feature/login
Switched to a new branch 'feature/login'
irzahameed@ubunto:~/DevOps_Lab2_Practice$ echo "login feature" >> login.txt
irzahameed@ubunto:~/DevOps_Lab2_Practice$ git add login.txt
irzahameed@ubunto:~/DevOps_Lab2_Practice$ git commit -m "Add login feature"
[feature/login be6915a] Add login feature
 1 file changed, 1 insertion(+)
 create mode 100644 login.txt
irzahameed@ubunto:~/DevOps_Lab2_Practice$ git checkout develop
Switched to branch 'develop'
irzahameed@ubunto:~/DevOps_Lab2_Practice$ git merge feature/login
Updating 9b73f4b..be6915a
Fast-forward
 login.txt | 1 +
 1 file changed, 1 insertion(+)
 create mode 100644 login.txt
irzahameed@ubunto:~/DevOps_Lab2_Practice$ git push origin develop
Enumerating objects: 4, done.
Counting objects: 100% (4/4), done.
Compressing objects: 100% (2/2), done.
Writing objects: 100% (3/3), 356 bytes | 118.00 KiB/s, done.
Total 3 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)
To github.com:irzahameed/DevOps_Lab2_Practice.git
   9b73f4b..be6915a  develop -> develop
irzahameed@ubunto:~/DevOps_Lab2_Practice$ git checkout staging
Switched to branch 'staging'
irzahameed@ubunto:~/DevOps_Lab2_Practice$ git merge develop
Updating 9b73f4b..be6915a
Fast-forward
 login.txt | 1 +
 1 file changed, 1 insertion(+)
 create mode 100644 login.txt
irzahameed@ubunto:~/DevOps_Lab2_Practice$ git push origin staging
Total 0 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)
To github.com:irzahameed/DevOps_Lab2_Practice.git
   9b73f4b..be6915a  staging -> staging
irzahameed@ubunto:~/DevOps_Lab2_Practice$ _
```

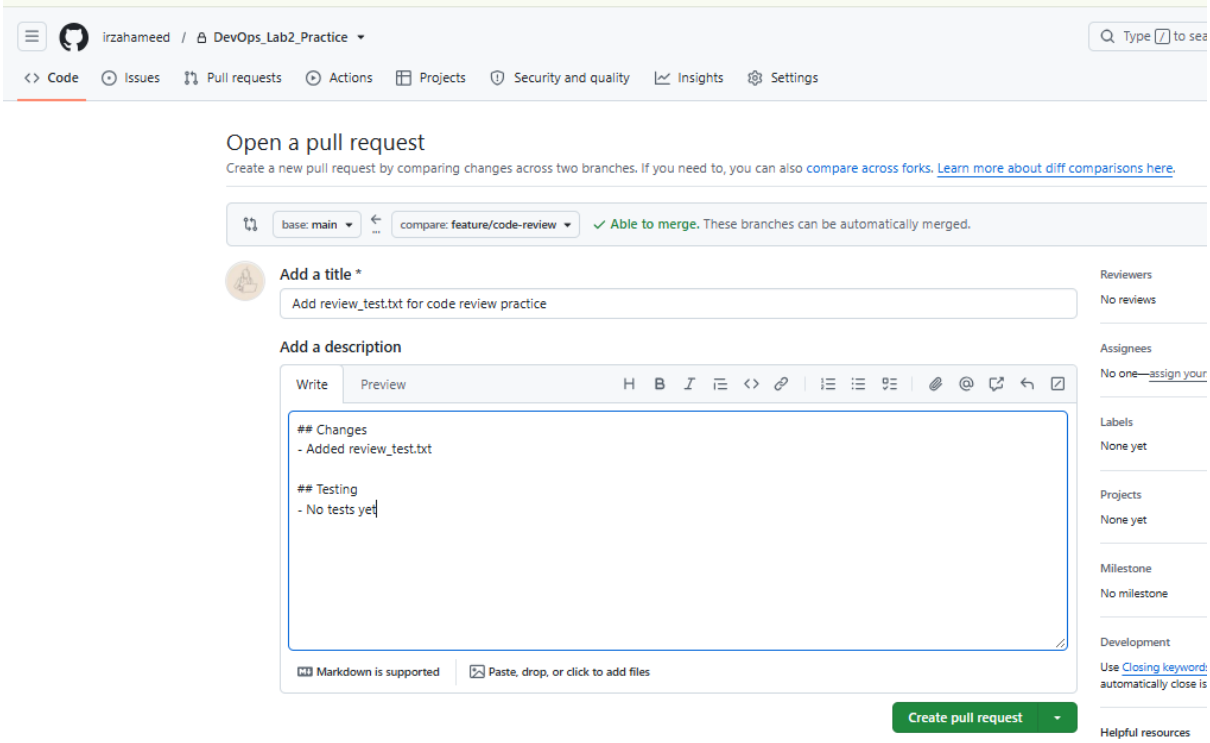
- Final merge into main (final_merge.png)

```
fatal: a branch named 'staging' already exists
irzahameed@ubunto:~/DevOps_Lab2_Practice$ git checkout main
Already on 'main'
Your branch is up to date with 'origin/main'.
irzahameed@ubunto:~/DevOps_Lab2_Practice$ git pull origin main
From github.com:irzahameed/DevOps_Lab2_Practice
 * branch            main       -> FETCH_HEAD
Already up to date.
irzahameed@ubunto:~/DevOps_Lab2_Practice$ git merge staging
Already up to date.
irzahameed@ubunto:~/DevOps_Lab2_Practice$ git push origin main
Everything up-to-date
irzahameed@ubunto:~/DevOps_Lab2_Practice$ _
```

Task 12: Code Review Workflow

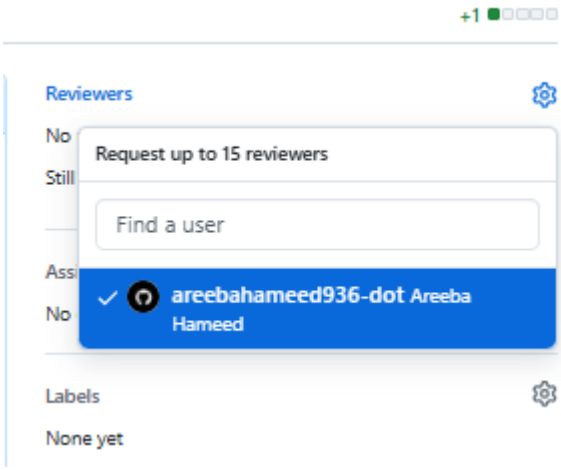
1. Create a Pull Request (PR) / Merge Request (MR):

- From a feature branch into main.
- Add a clear title and description summarizing the changes.
- Screenshot as pr_create_details.png.



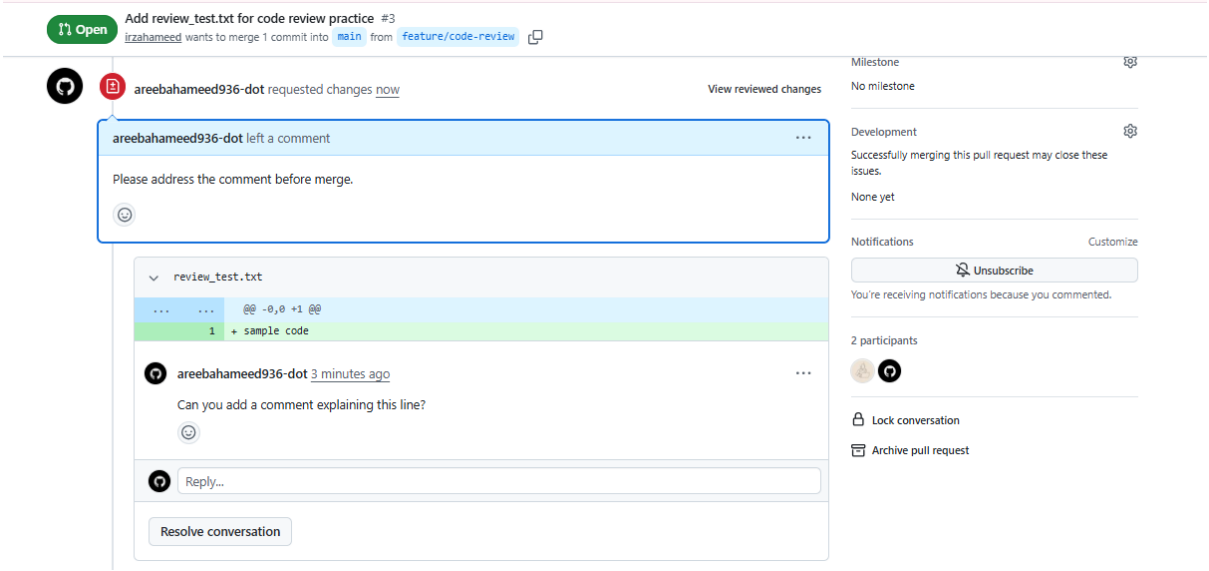
2. Assign a reviewer (teammate or second account).

- Screenshot as pr_assigned_reviewer.png.



3. Reviewer Actions (capture screenshots for each):

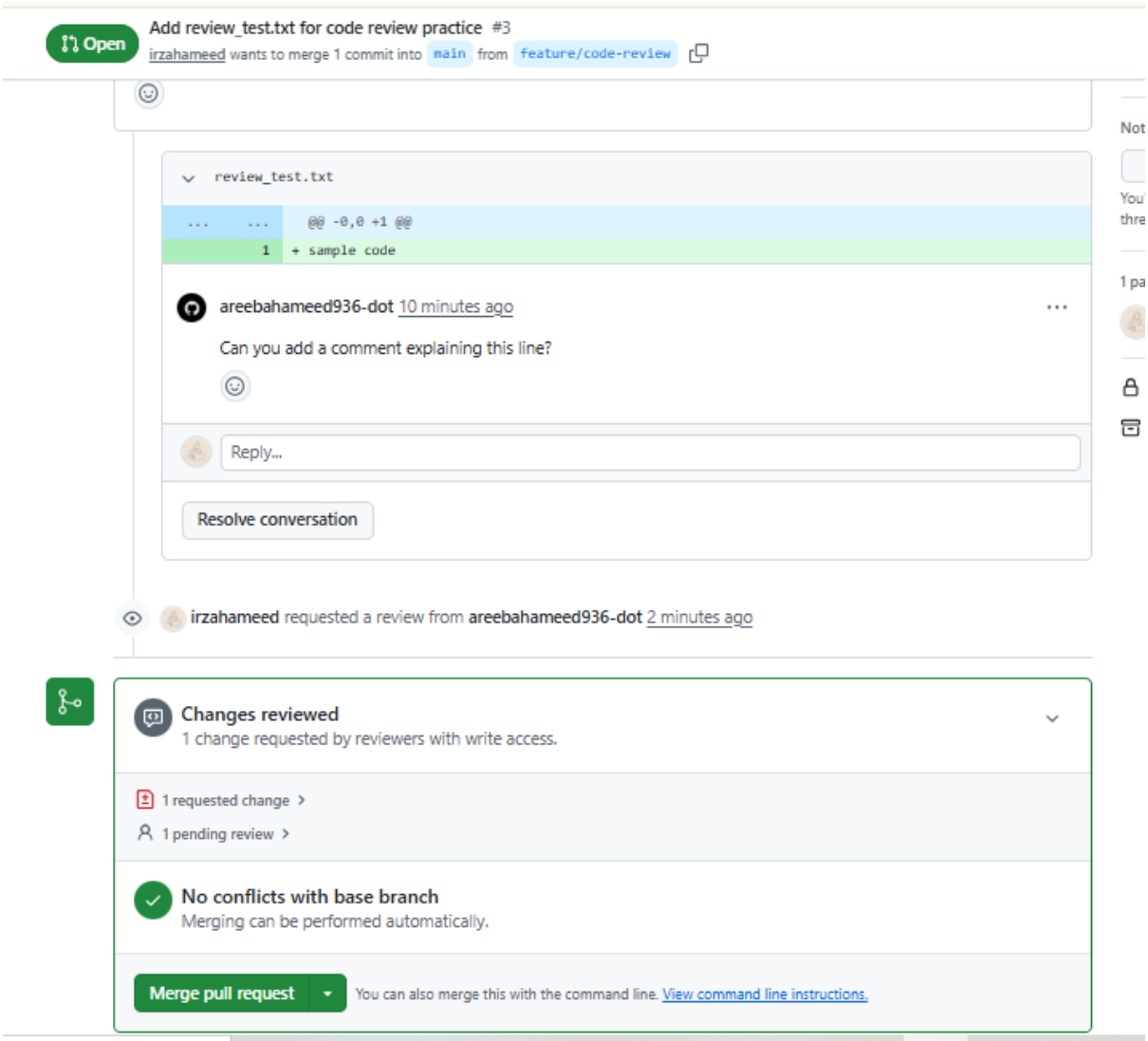
- Request changes and add comments → pr_request_changes.png



- Reject or close a practice PR → pr_rejected.png. Use a separate practice PR if necessary so all required review states can be demonstrated safely.

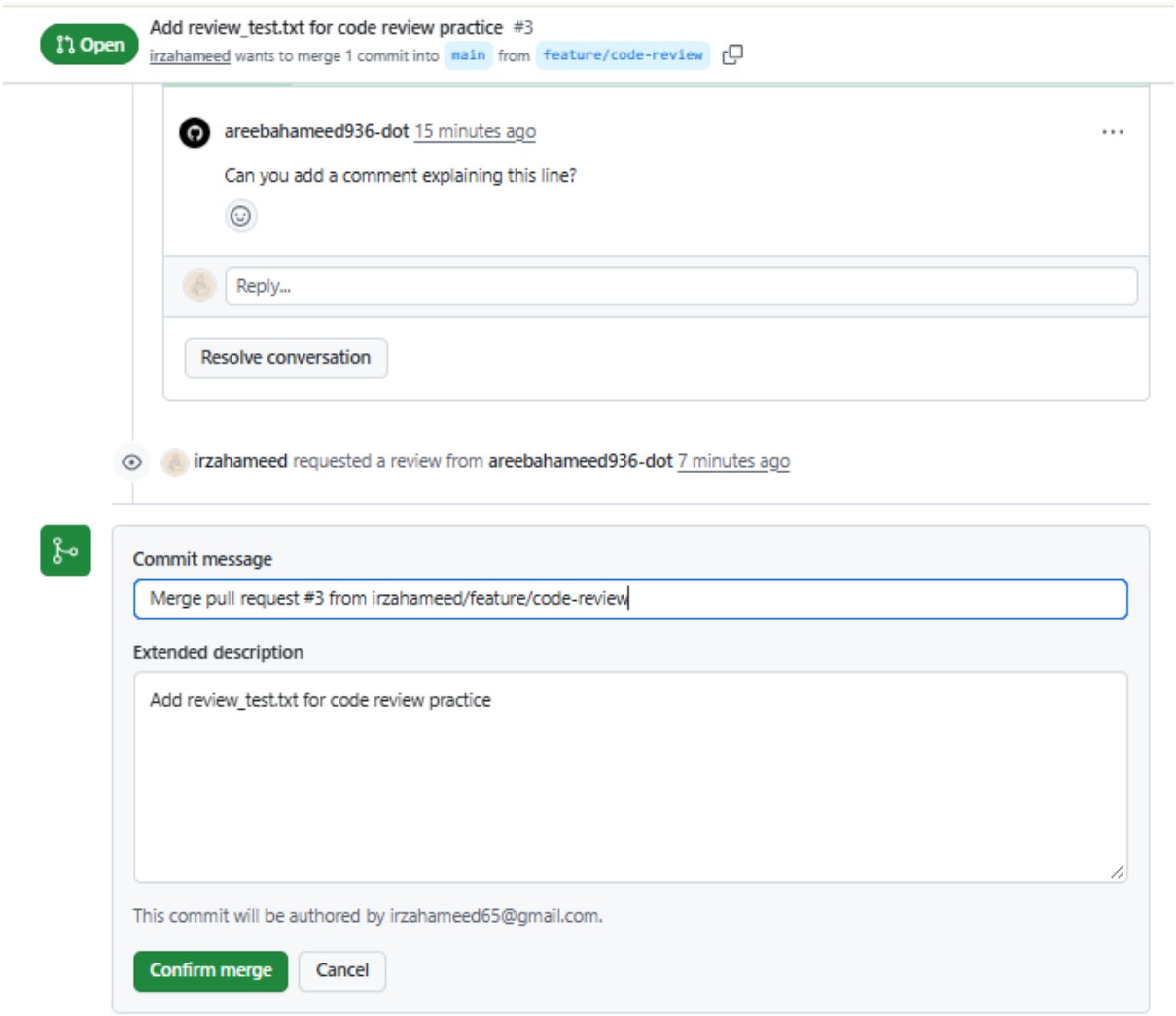
4. Author Actions:

- Fix issues in the same branch and push new commits.
- PR updates automatically.
- Screenshot updated PR with new commits → pr_updated_with_commits.png.

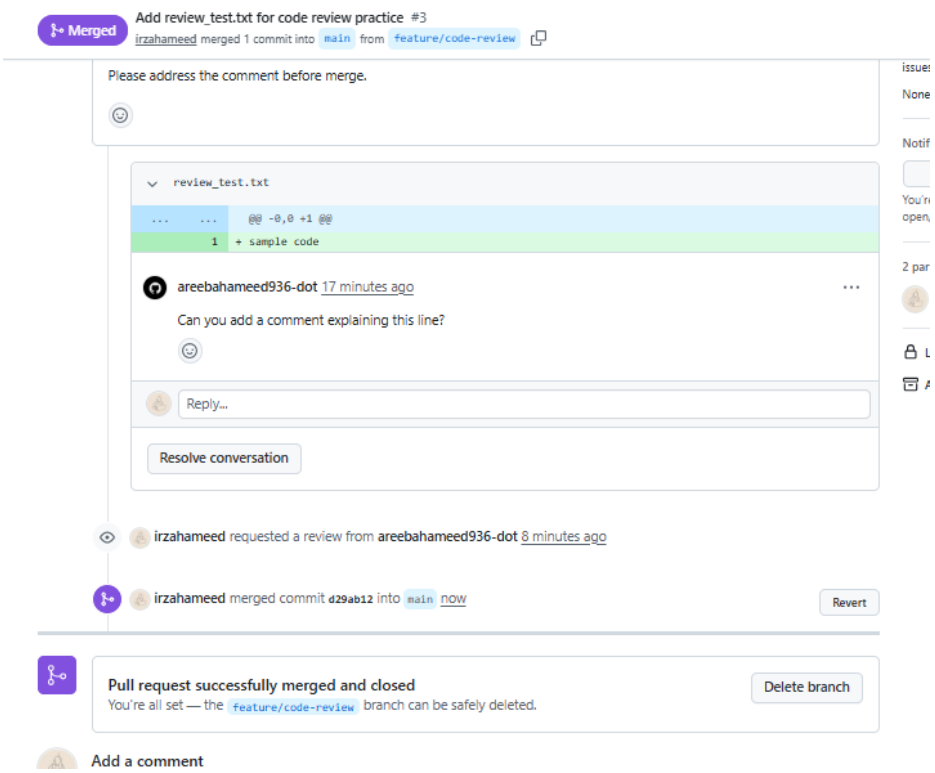


5. Merge the PR:

- Reviewer approves → merge using GitHub/GitLab UI.
- Choose merge style (merge commit, squash, or rebase).
- Screenshot confirmation → pr_merge_confirm.png.

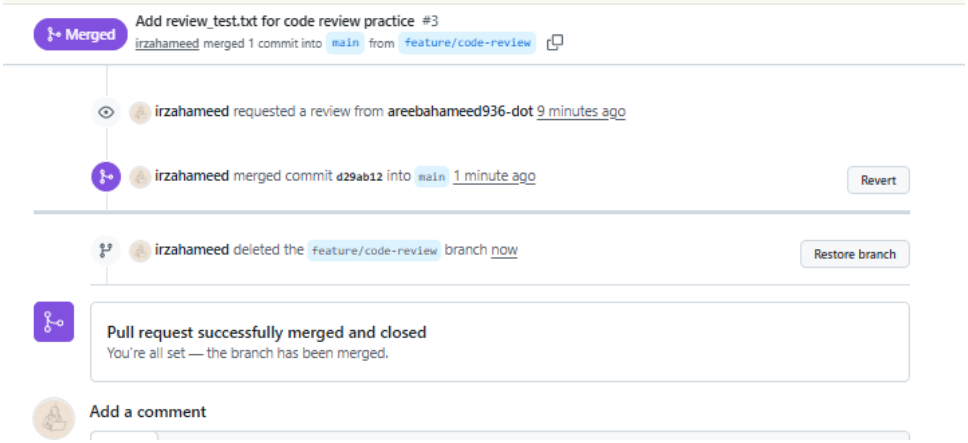


- Screenshot result → pr_merged.png.



6. Branch Handling After Merge:

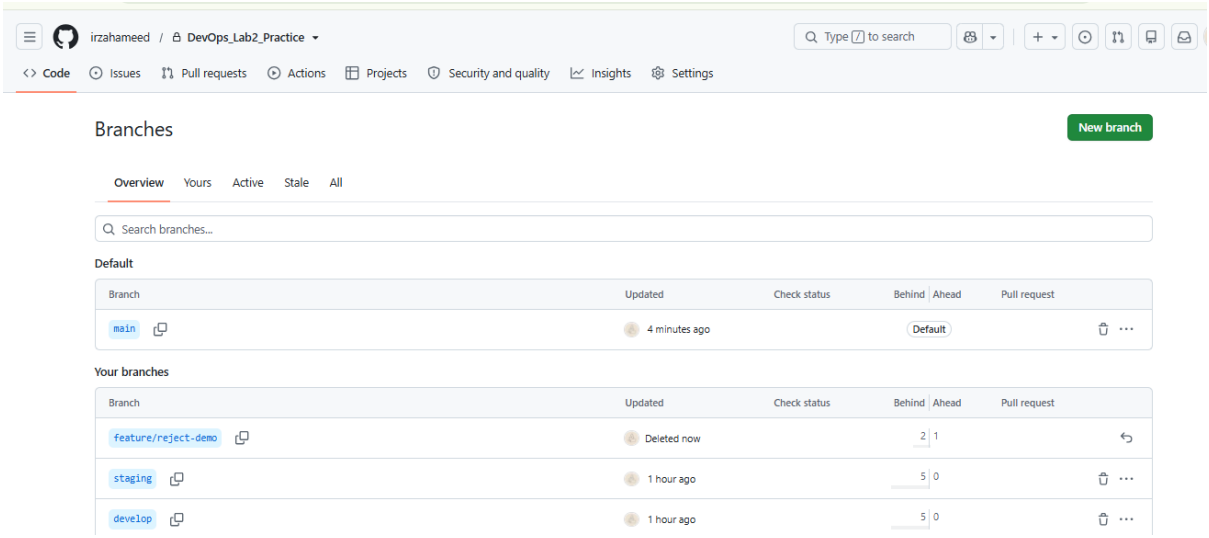
- Delete the merged PR branch and save the evidence as pr_branch_deleted.png.



Task 13: Branch Cleanup Best Practices

1. Delete Remote Branch After Merge:

- Delete a merged branch through the GitHub UI and save the evidence as remote_branch_deleted.png.



- Delete another completed remote branch from the terminal:

git push origin --delete <branch-name>

- Save the terminal evidence as remote_branch_delete_cmd.png.

```
irzahameed@ubunto:~/DevOps_Lab2_Practice$ git push origin --delete feature-1
To github.com:irzahameed/DevOps_Lab2_Practice.git
 - [deleted]         feature-1
irzahameed@ubunto:~/DevOps_Lab2_Practice$
```

2. Update Local Repository:

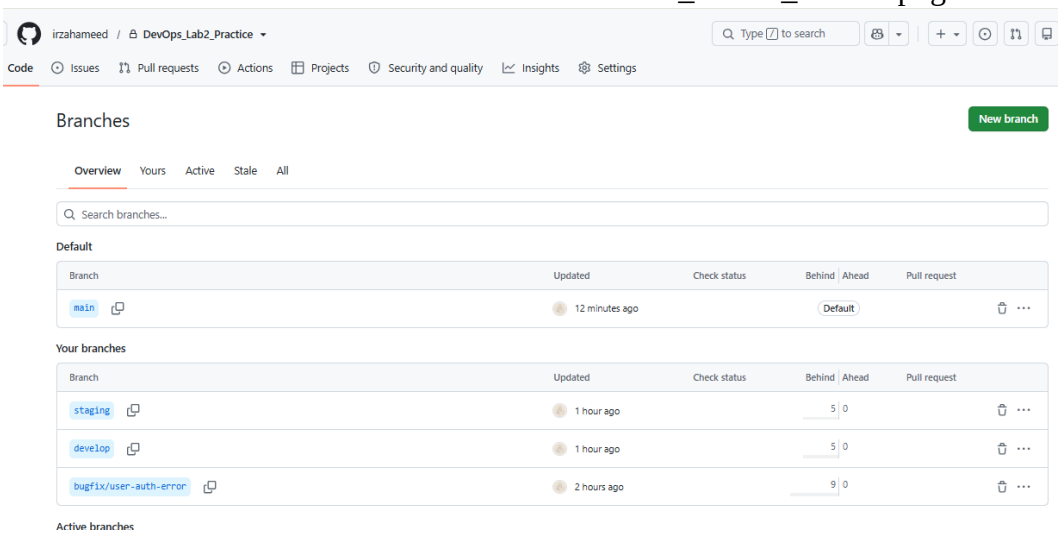
git checkout main

git pull origin main

git branch -d <branch-name>

git branch

3. Save the GitHub branch-deletion evidence as remote_branch_deleted.png.



4. Save the terminal command used to delete a remote branch as remote_branch_delete_cmd.png.

